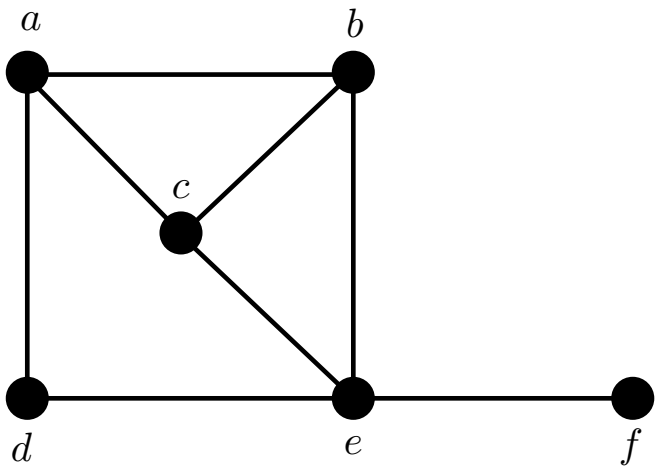
ISyE/Math/CS 425
Introduction to Combinatorial Optimization

2.1 Minimum Spanning Trees

Prof. Carla Michini
University of Wisconsin-Madison

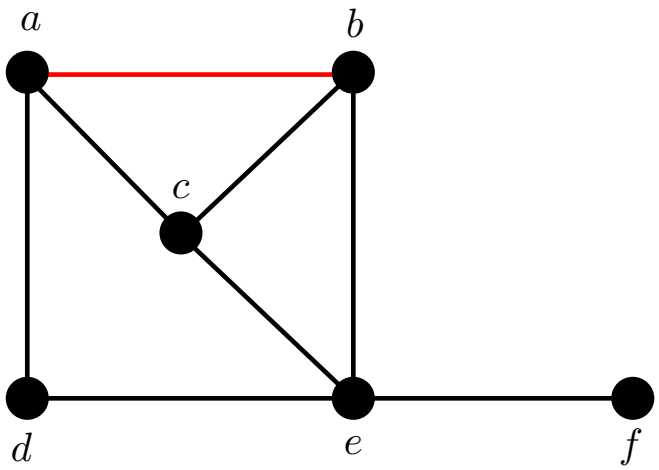
2.1 Minimum Spanning Trees

Graphs, subgraphs, paths



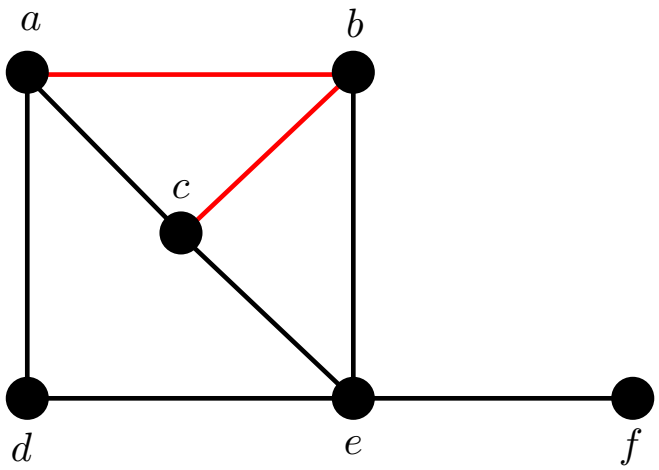
- A graph $G = (V, E)$ consists of
  - a set $V = V(G)$ of nodes,
  - a set $E = E(G)$ of edges.
- Each edge $e \in E$ consists of an unordered pair of distinct nodes, its ends, we write $e = \{v, w\} = vw$.
- An edge e is incident to a node v if $v \in e$.
- Two nodes v, w are adjacent, if vw is an edge.
- Two edges $e \neq f$ are adjacent if they have an end in common.

Graphs, subgraphs, paths



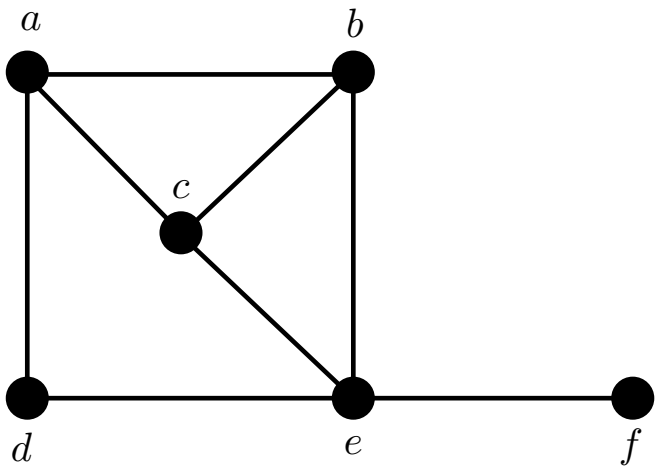
- A graph $G = (V, E)$ consists of
  - a set $V = V(G)$ of nodes,
  - a set $E = E(G)$ of edges.
- Each edge $e \in E$ consists of an unordered pair of distinct nodes, its ends, we write $e = \{v, w\} = vw$.
- An edge e is incident to a node v if $v \in e$.
- Two nodes v, w are adjacent, if vw is an edge.
- Two edges $e \neq f$ are adjacent if they have an end in common.

Graphs, subgraphs, paths



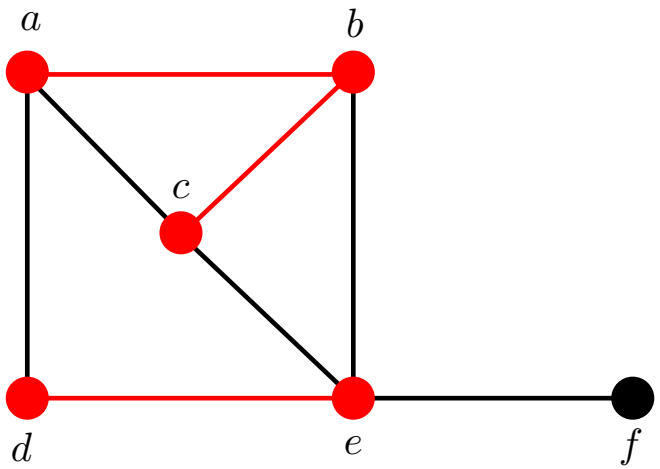
- A graph $G = (V, E)$ consists of
  - a set $V = V(G)$ of nodes,
  - a set $E = E(G)$ of edges.
- Each edge $e \in E$ consists of an unordered pair of distinct nodes, its ends, we write $e = \{v, w\} = vw$.
- An edge e is incident to a node v if $v \in e$.
- Two nodes v, w are adjacent, if vw is an edge.
- Two edges $e \neq f$ are adjacent if they have an end in common.

Graphs, subgraphs, paths



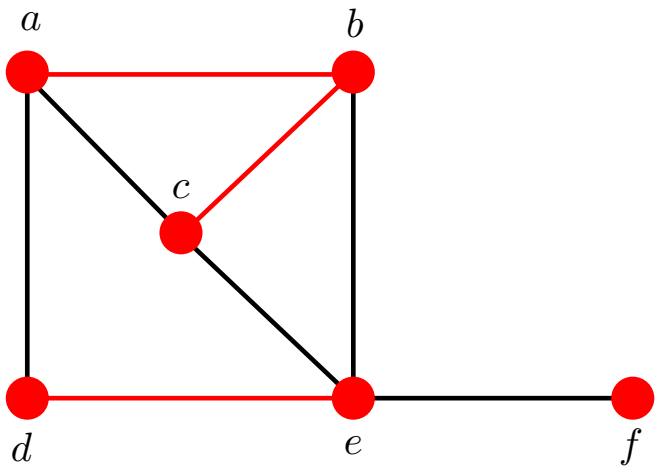
- A subgraph H of G is a graph such that $V(H) \subseteq V(G)$, and $E(H) \subseteq E(G)$.
- A subgraph H of G is spanning if $V(H) = V(G)$.

Graphs, subgraphs, paths



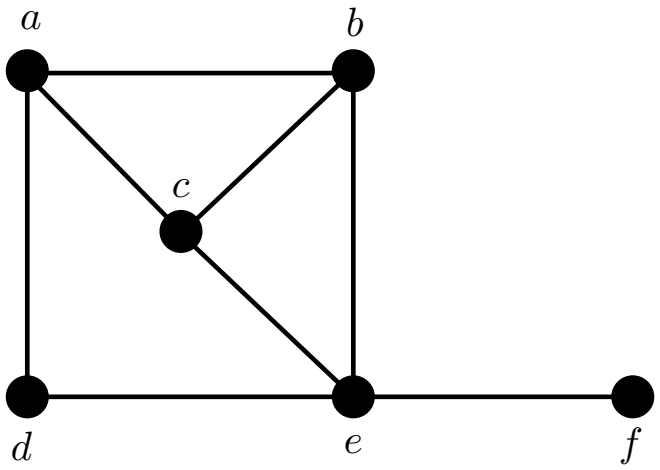
- A subgraph H of G is a graph such that $V(H) \subseteq V(G)$, and $E(H) \subseteq E(G)$.
- A subgraph H of G is spanning if $V(H) = V(G)$.

Graphs, subgraphs, paths



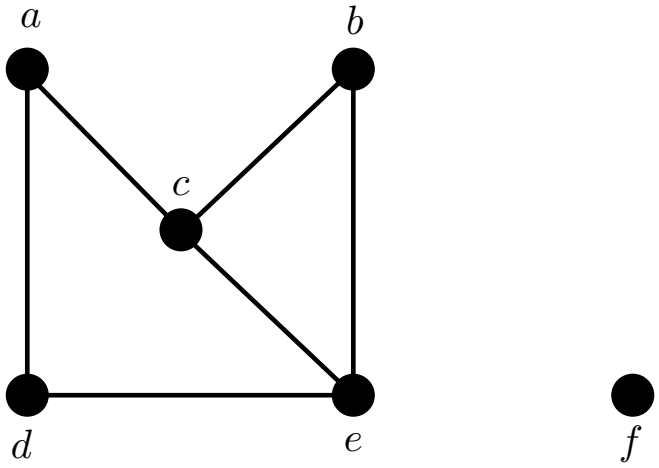
- A subgraph H of G is a graph such that $V(H) \subseteq V(G)$, and $E(H) \subseteq E(G)$.
- A subgraph H of G is spanning if $V(H) = V(G)$.

Graphs, subgraphs, paths



- For $F \subseteq E$, $G \setminus F$ denotes the subgraph H obtained by deleting F, that is, $V(H) = V$ and $E(H) = E \setminus F$.
- For $U \subseteq V$, $G \setminus U$ denotes the subgraph H obtained by deleting U, that is, $V(H) = V \setminus U$ and $E(H) = E \setminus F$, where F is the set of all edges incident with nodes in U.
- $G \setminus U$ may be referred to as the subgraph $G[V \setminus U]$ of G induced by $V \setminus U$.

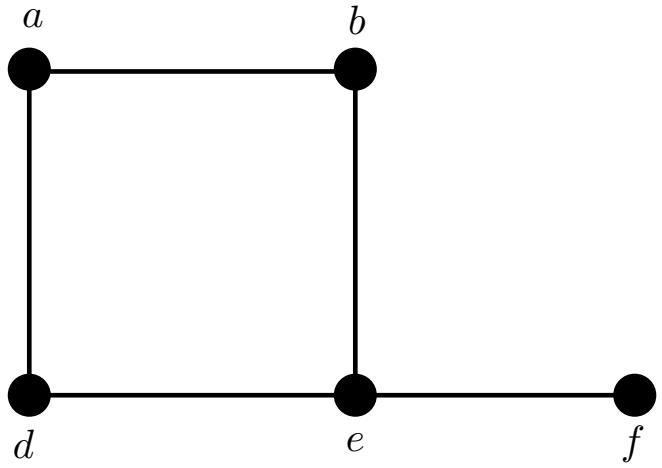
Graphs, subgraphs, paths



$G \setminus \{ab, ef\}$

- For $F \subseteq E$, $G \setminus F$ denotes the subgraph H obtained by deleting F, that is, $V(H) = V$ and $E(H) = E \setminus F$.
- For $U \subseteq V$, $G \setminus U$ denotes the subgraph H obtained by deleting U, that is, $V(H) = V \setminus U$ and $E(H) = E \setminus F$, where F is the set of all edges incident with nodes in U.
- $G \setminus U$ may be referred to as the subgraph $G[V \setminus U]$ of G induced by $V \setminus U$.

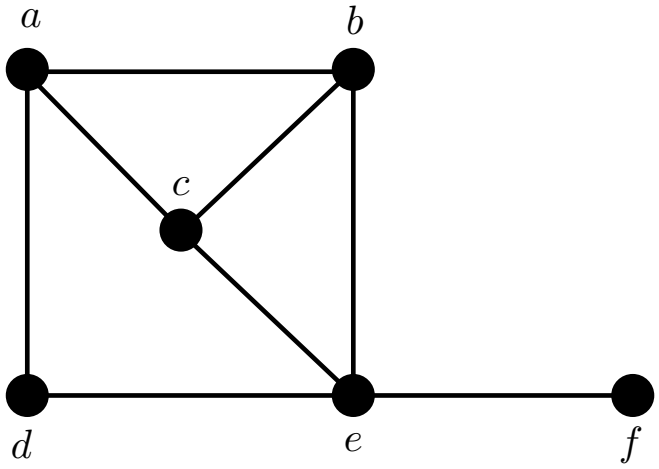
Graphs, subgraphs, paths



$G \setminus \{c\}$

- For $F \subseteq E$, $G \setminus F$ denotes the subgraph H obtained by deleting F, that is, $V(H) = V$ and $E(H) = E \setminus F$.
- For $U \subseteq V$, $G \setminus U$ denotes the subgraph H obtained by deleting U, that is, $V(H) = V \setminus U$ and $E(H) = E \setminus F$, where F is the set of all edges incident with nodes in U.
- $G \setminus U$ may be referred to as the subgraph $G[V \setminus U]$ of G induced by $V \setminus U$.

Graphs, subgraphs, paths

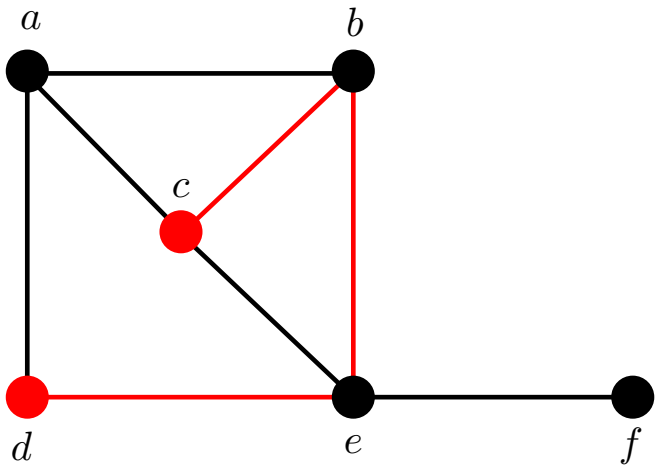


- A path P in a graph G is a sequence

$$P = v_0, e_1, v_1, \ldots, e_k, v_k$$

  where $e_i = v_{i-1}v_i$ for $1 \leq i \leq k$.
- We often just write $P = v_0, v_1, \ldots, v_k$.
- We say that P is from v_0 to v_k, or that it is a (v_0, v_k)-path.
- We say that the length of P is k.
- G is connected if every pair of nodes is joined by a path.

Graphs, subgraphs, paths



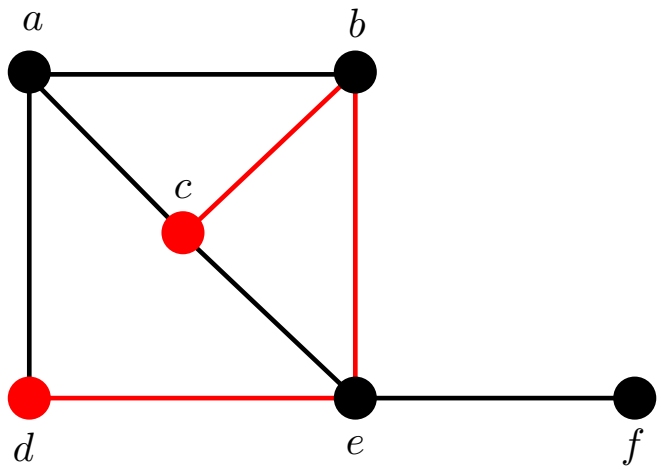
A (c, d)-path:
$P = d, e, b, c$

- A path P in a graph G is a sequence

$$P = v_0, e_1, v_1, \ldots, e_k, v_k$$

  where $e_i = v_{i-1}v_i$ for $1 \leq i \leq k$.
- We often just write $P = v_0, v_1, \ldots, v_k$.
- We say that P is from v_0 to v_k, or that it is a (v_0, v_k)-path.
- We say that the length of P is k.
- G is connected if every pair of nodes is joined by a path.

Graphs, subgraphs, paths

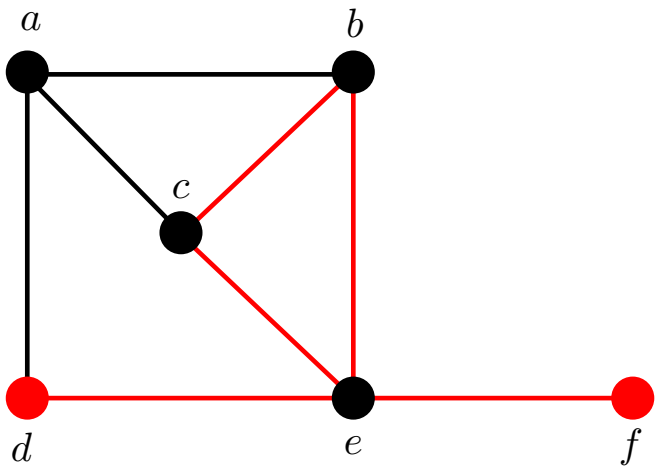


A (c, d)-path:
$P = d, e, b, c$

$$P = v_0, e_1, v_1, \ldots, e_k, v_k$$

- P is closed if $v_0 = v_k$.
- P is edge-simple if $e_1, \ldots, e_k$ are distinct.
- P is simple if $v_0, \ldots, v_k$ are distinct.
- P is a circuit (or cycle) if it is edge-simple and closed, $v_0, \ldots, v_{k-1}$ are distinct, and $k \geq 1$.

Graphs, subgraphs, paths

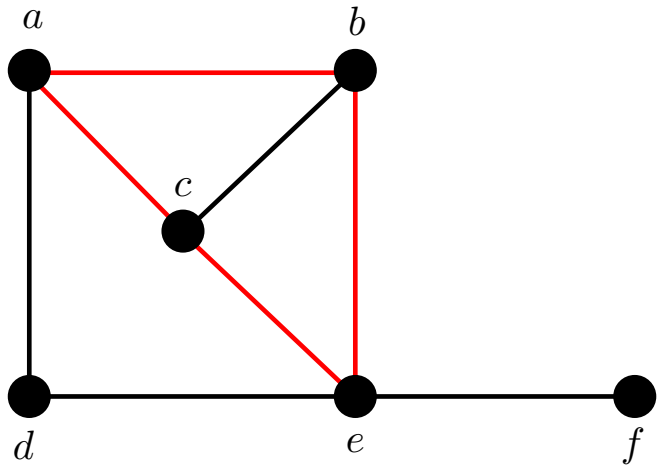


A (d, f)-path:
$P = d, e, b, c, e, f$

$$P = v_0, e_1, v_1, \ldots, e_k, v_k$$

- P is closed if $v_0 = v_k$.
- P is edge-simple if $e_1, \ldots, e_k$ are distinct.
- P is simple if $v_0, \ldots, v_k$ are distinct.
- P is a circuit (or cycle) if it is edge-simple and closed, $v_0, \ldots, v_{k-1}$ are distinct, and $k \geq 1$.

Graphs, subgraphs, paths

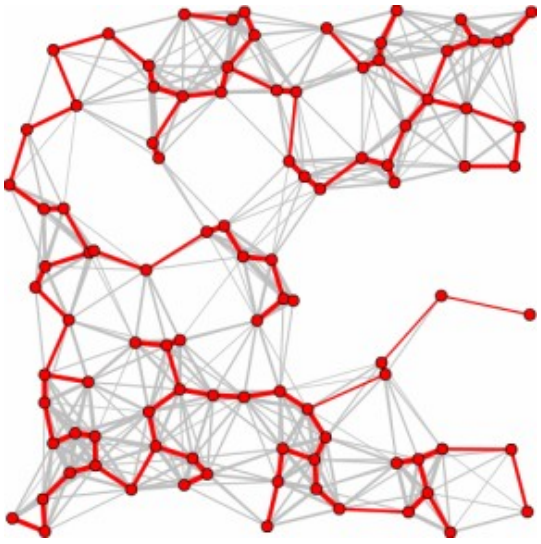


A circuit: $C = a, b, e, c, a$

$$P = v_0, e_1, v_1, \ldots, e_k, v_k$$

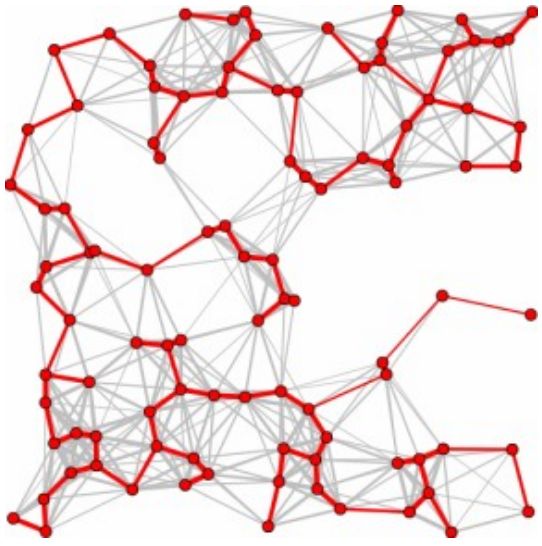
- P is closed if $v_0 = v_k$.
- P is edge-simple if $e_1, \ldots, e_k$ are distinct.
- P is simple if $v_0, \ldots, v_k$ are distinct.
- P is a circuit (or cycle) if it is edge-simple and closed, $v_0, \ldots, v_{k-1}$ are distinct, and $k \geq 1$.

Connector problem



- A company has a number of offices and wants to design a network linking them.
- For certain pairs v, w of offices, a link joining v and w of positive cost c_{vw} can be built.
- Problem: construct links so that every pair of offices can communicate, minimizing the total construction cost.

Connector problem



- A company has a number of offices and wants to design a network linking them.
- For certain pairs v, w of offices, a link joining v and w of positive cost c_{vw} can be built.
- Problem: construct links so that every pair of offices can communicate, minimizing the total construction cost.

Given a connected graph G and a positive cost c_e, $\forall e \in E$, find a minimum-cost spanning connected subgraph of G.
(Where the cost of a subgraph is the sum of the costs of its edges.)

Structure of optimal subgraphs

Lemma 2.1
An edge $e = uv$ of G is an edge of a circuit of G if and only if there is a path in $G \setminus e$ from u to v.

Let's prove it!

Structure of optimal subgraphs

Lemma 2.1
An edge $e = uv$ of G is an edge of a circuit of G if and only if there is a path in $G \setminus e$ from u to v.

Let's prove it!

Thus, if we delete an edge of a circuit from a connected graph, the new graph is still connected. Why?

Reduction to MST problem

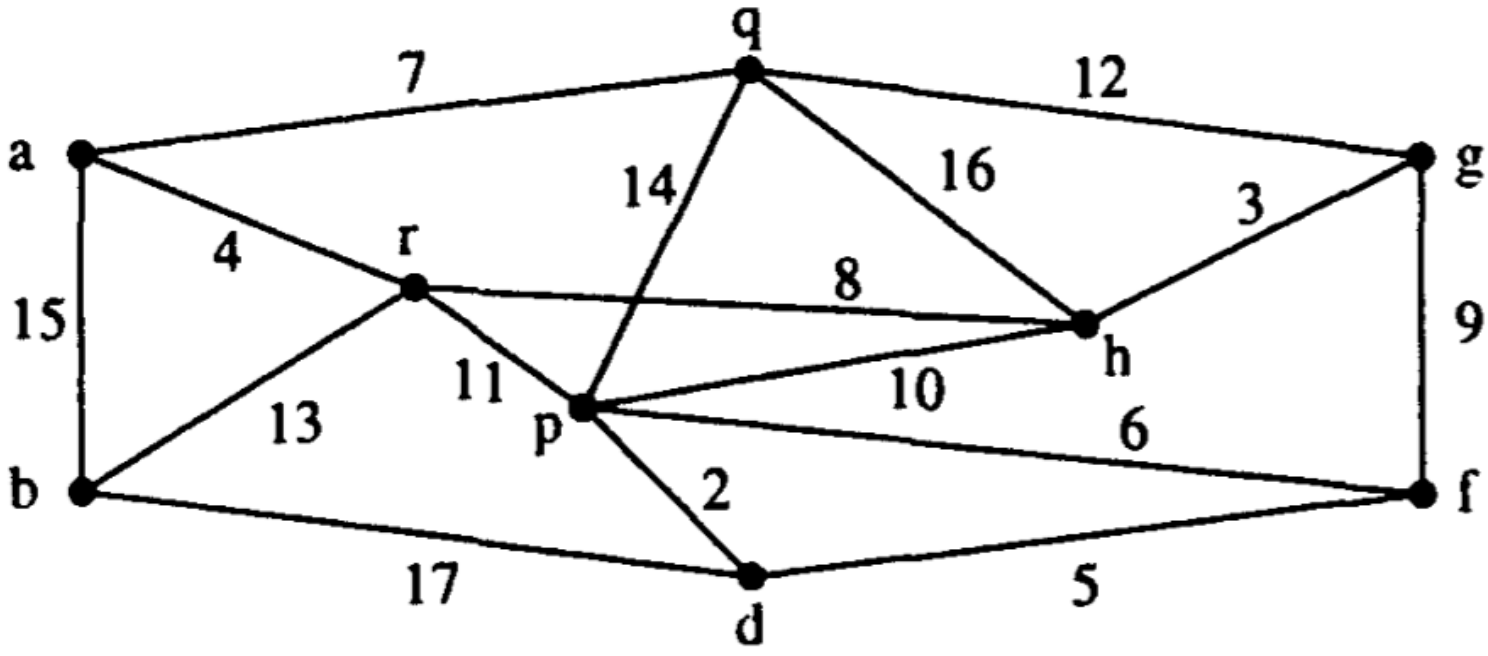
- So an optimal solution to the connector problem will not have any circuits.
- A graph having no circuit is called a forest; a connected forest is called a tree.
- We can solve the connector problem by solving the following:

Minimum spanning tree (MST) problem
Given a connected graph G and a real cost c_e, $\forall e \in E$, find a minimum cost spanning tree of G.

- Connector problem and MST problem are equivalent for positive edge costs.
- We solve the MST problem for arbitrary edge-costs.

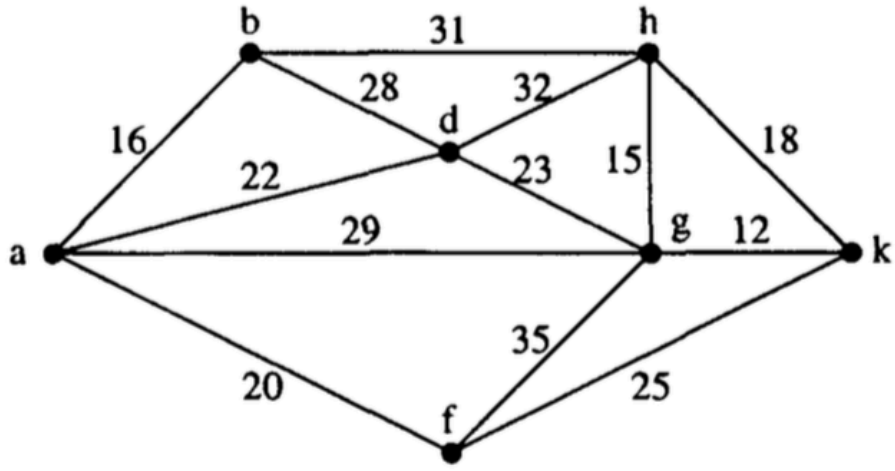
Exercise

Let's try to solve a MST problem!



Kruskal's algorithm for MST

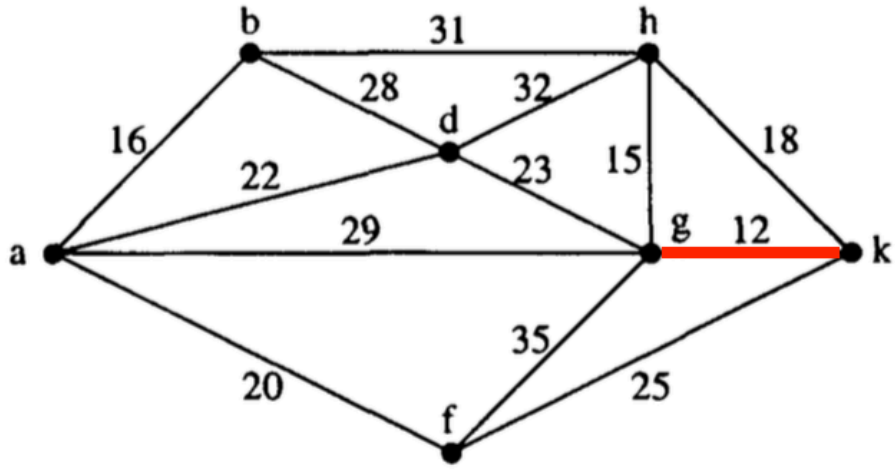
1. Initialize $H = (V, F)$ as $(V, \emptyset)$.
2. At each step add to F a least-cost edge $e \notin F$ such that H remains a (spanning) forest.
3. Stop when H is a (spanning) tree.



At each step the algorithm maintains a spanning forest. At the end the spanning forest is connected (a tree).

Kruskal's algorithm for MST

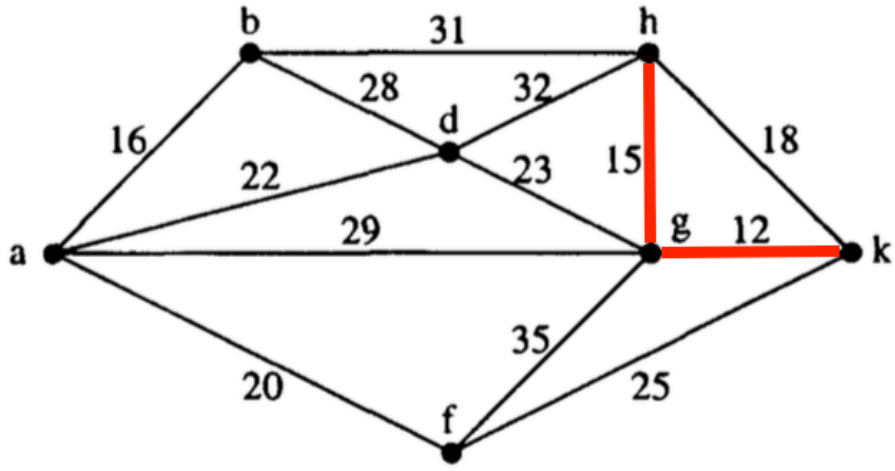
1. Initialize $H = (V, F)$ as $(V, \emptyset)$.
2. At each step add to F a least-cost edge $e \notin F$ such that H remains a (spanning) forest.
3. Stop when H is a (spanning) tree.



At each step the algorithm maintains a spanning forest. At the end the spanning forest is connected (a tree).

Kruskal's algorithm for MST

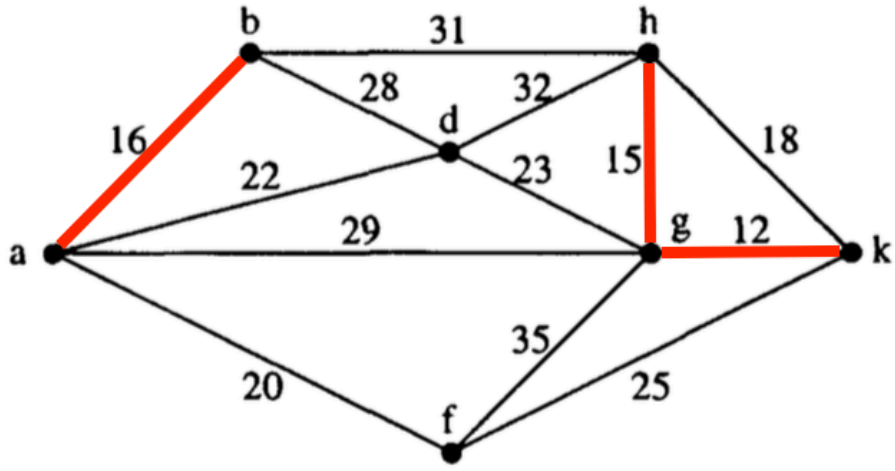
1. Initialize $H = (V, F)$ as $(V, \emptyset)$.
2. At each step add to F a least-cost edge $e \notin F$ such that H remains a (spanning) forest.
3. Stop when H is a (spanning) tree.



At each step the algorithm maintains a spanning forest. At the end the spanning forest is connected (a tree).

Kruskal's algorithm for MST

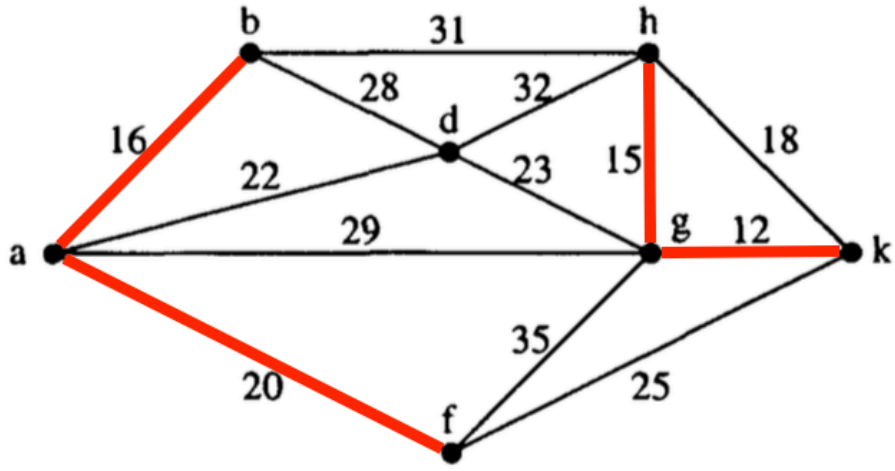
1. Initialize $H = (V, F)$ as $(V, \emptyset)$.
2. At each step add to F a least-cost edge $e \notin F$ such that H remains a (spanning) forest.
3. Stop when H is a (spanning) tree.



At each step the algorithm maintains a spanning forest. At the end the spanning forest is connected (a tree).

Kruskal's algorithm for MST

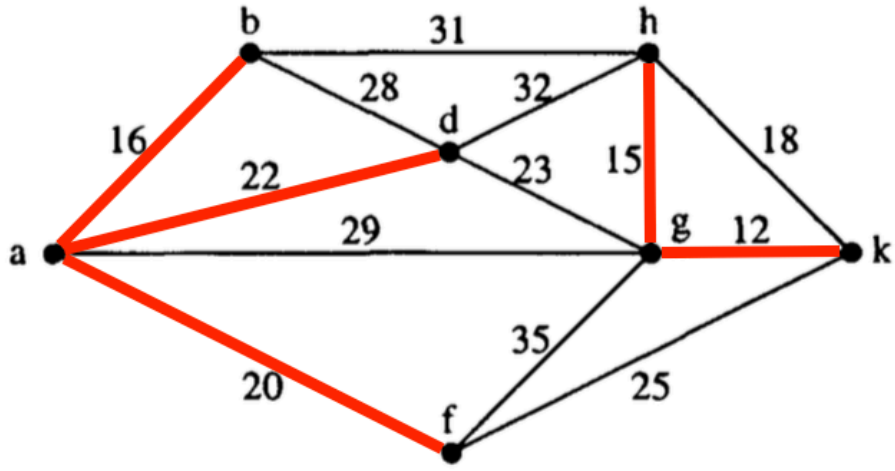
1. Initialize $H = (V, F)$ as $(V, \emptyset)$.
2. At each step add to F a least-cost edge $e \notin F$ such that H remains a (spanning) forest.
3. Stop when H is a (spanning) tree.



At each step the algorithm maintains a spanning forest. At the end the spanning forest is connected (a tree).

Kruskal's algorithm for MST

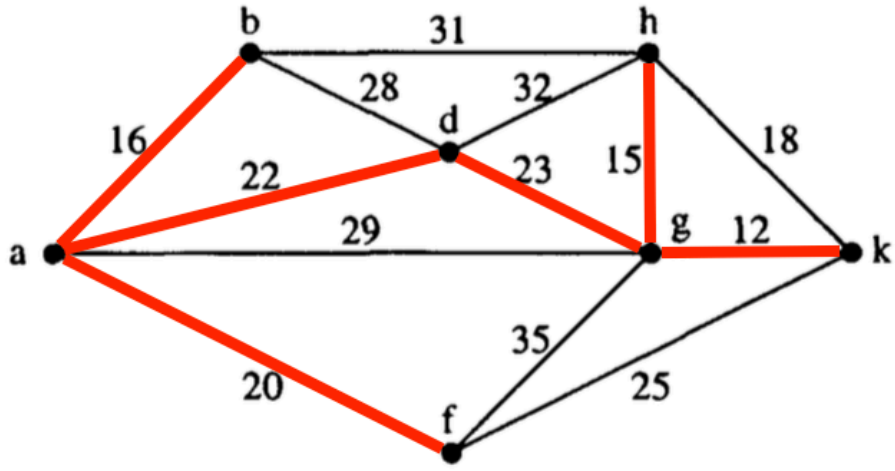
1. Initialize $H = (V, F)$ as $(V, \emptyset)$.
2. At each step add to F a least-cost edge $e \notin F$ such that H remains a (spanning) forest.
3. Stop when H is a (spanning) tree.



At each step the algorithm maintains a spanning forest. At the end the spanning forest is connected (a tree).

Kruskal's algorithm for MST

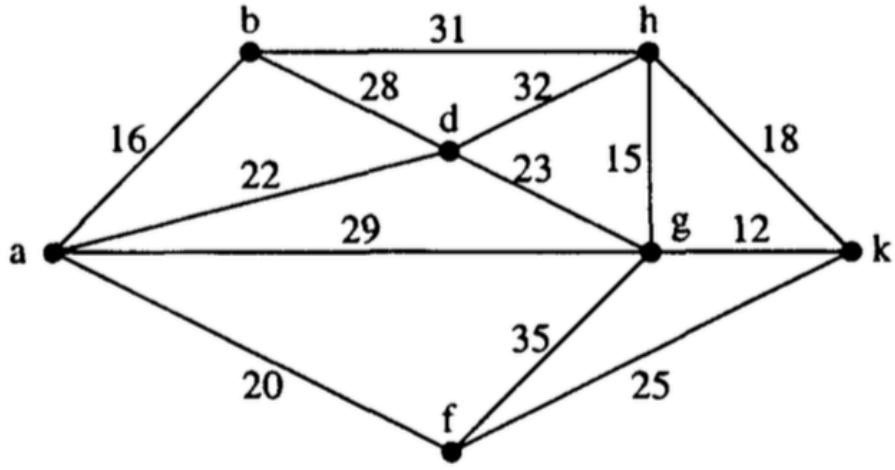
1. Initialize $H = (V, F)$ as $(V, \emptyset)$.
2. At each step add to F a least-cost edge $e \notin F$ such that H remains a (spanning) forest.
3. Stop when H is a (spanning) tree.



At each step the algorithm maintains a spanning forest. At the end the spanning forest is connected (a tree).

Prim's algorithm for MST

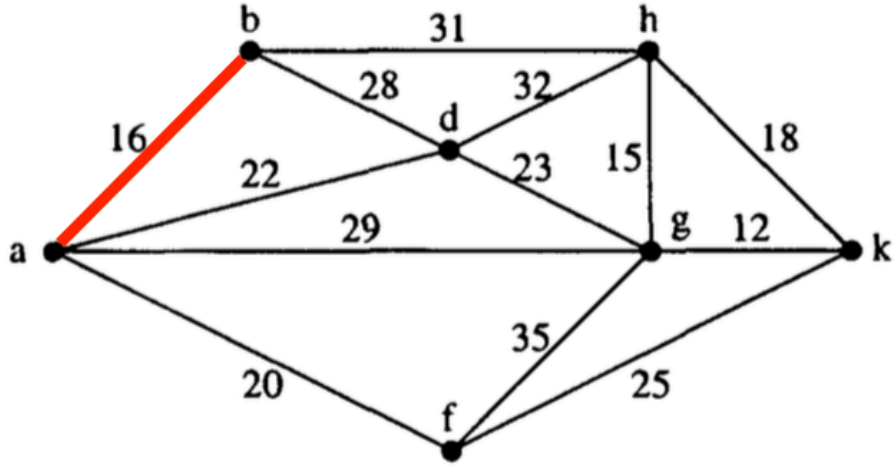
1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$ for some $r \in V$.
2. At each step add to T a least-cost edge $e \notin T$ such that H remains a tree.
3. Stop when H is spanning (tree).



At each step the algorithm maintains a tree. At the end the tree is spanning.

Prim's algorithm for MST

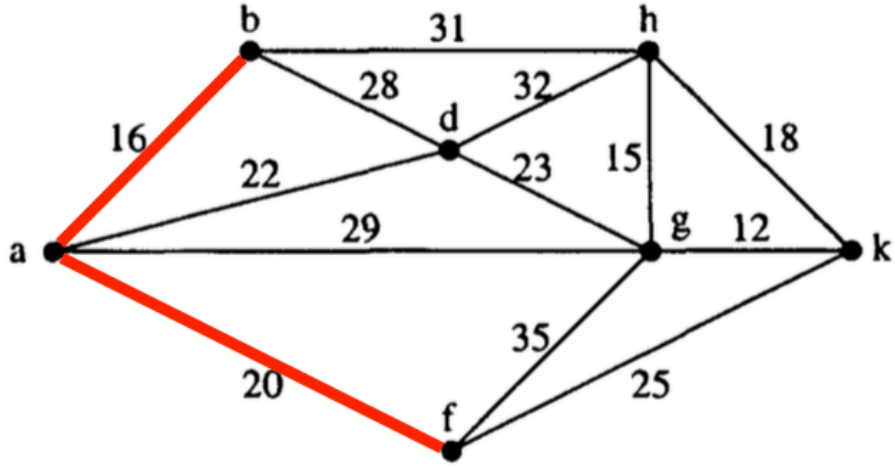
1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$ for some $r \in V$.
2. At each step add to T a least-cost edge $e \notin T$ such that H remains a tree.
3. Stop when H is spanning (tree).



At each step the algorithm maintains a tree. At the end the tree is spanning.

Prim's algorithm for MST

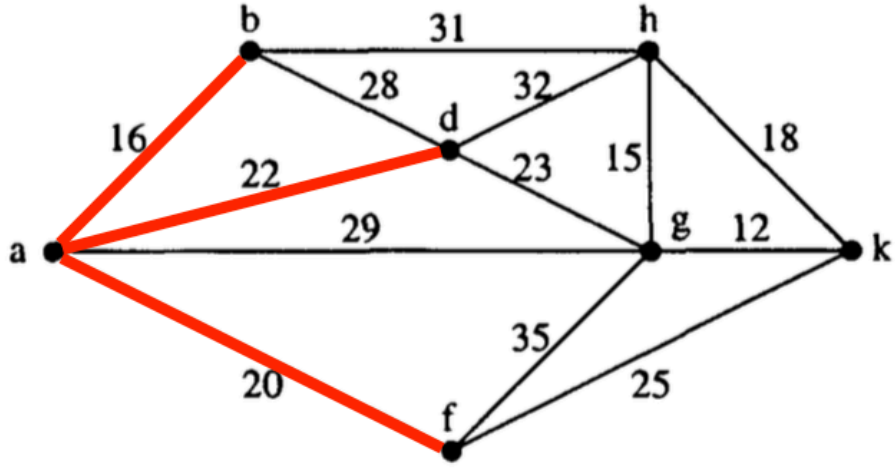
1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$ for some $r \in V$.
2. At each step add to T a least-cost edge $e \notin T$ such that H remains a tree.
3. Stop when H is spanning (tree).



At each step the algorithm maintains a tree. At the end the tree is spanning.

Prim's algorithm for MST

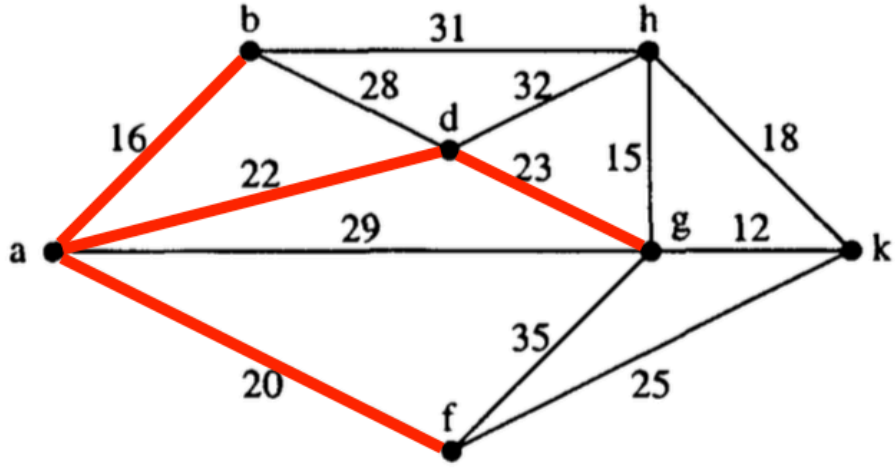
1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$ for some $r \in V$.
2. At each step add to T a least-cost edge $e \notin T$ such that H remains a tree.
3. Stop when H is spanning (tree).



At each step the algorithm maintains a tree. At the end the tree is spanning.

Prim's algorithm for MST

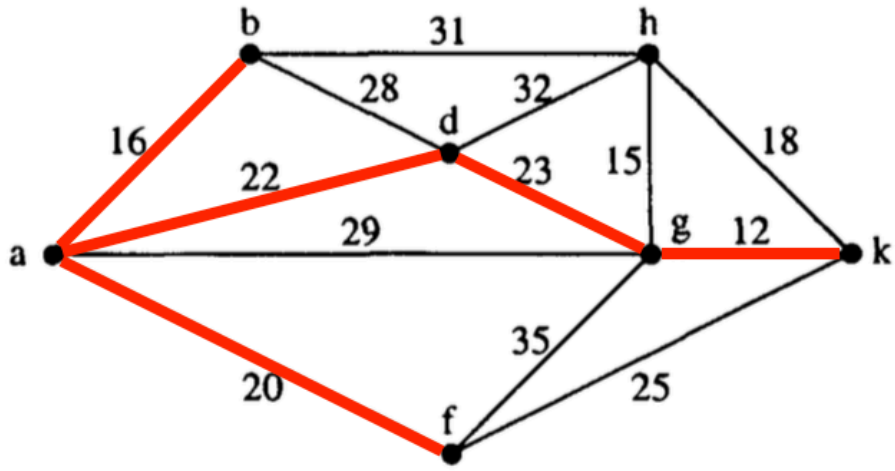
1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$ for some $r \in V$.
2. At each step add to T a least-cost edge $e \notin T$ such that H remains a tree.
3. Stop when H is spanning (tree).



At each step the algorithm maintains a tree. At the end the tree is spanning.

Prim's algorithm for MST

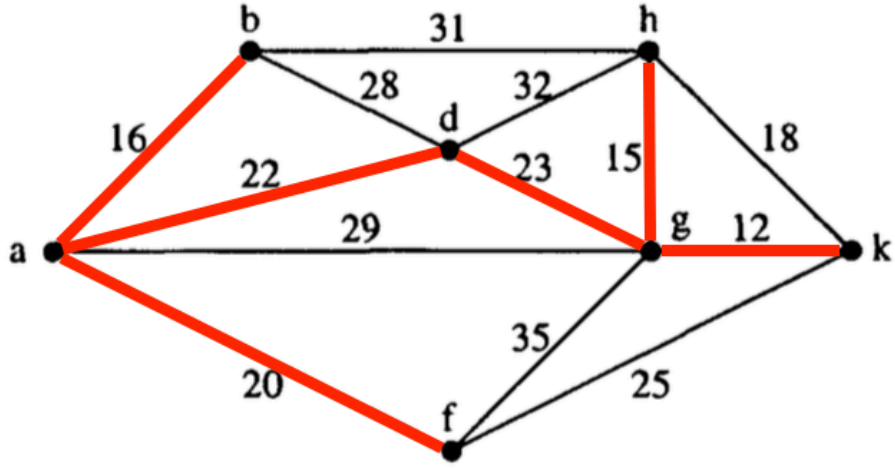
1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$ for some $r \in V$.
2. At each step add to T a least-cost edge $e \notin T$ such that H remains a tree.
3. Stop when H is spanning (tree).



At each step the algorithm maintains a tree. At the end the tree is spanning.

Prim's algorithm for MST

1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$ for some $r \in V$.
2. At each step add to T a least-cost edge $e \notin T$ such that H remains a tree.
3. Stop when H is spanning (tree).



At each step the algorithm maintains a tree. At the end the tree is spanning.

We now show the correctness of both MST algorithms...

Cuts

- For a graph $G = (V, E)$ and $U \subseteq V$, we denote by $\delta(U) = \{e \in E : e = uv,\ u \in U, v \in V \setminus U\}$.
- $\delta(U)$ is called a cut of G.
- If $U = \emptyset$ or $U = V$, $\delta(U) = \emptyset$ is called a trivial cut
- Exercise: For each non-trivial cut $\delta(U)$, and for two nodes $u \in U$ and $v \in V \setminus U$, there is no (u, v)-path in $G \setminus \delta(U)$.

Intermediate results

Lemma 2.1
An edge $e = uv$ of G is an edge of a circuit of G if and only if there is a path in $G \setminus e$ from u to v.

Theorem 2.3
A graph $G = (V, E)$ is connected if and only if there is no set $U \subseteq V$, $\emptyset \neq U \neq V$, with $\delta(U) = \emptyset$.

Intermediate results

Lemma 2.7
Let $H = (V, T)$ be a spanning tree of G, let $e = vw$ be an edge of G but not H, and let f be an edge of a simple path in T from v to w. Then the subgraph $H' = (V, T \cup \{e\} \setminus \{f\})$ is a spanning tree of G.

A subset F of edges of G is extendible to a MST if F is contained in the edge-set of some MST of G.

Theorem 2.4
Suppose that $B \subseteq E$, that B is extendible to a MST, and that e is a minimum-cost edge of some cut D satisfying $D \cap B = \emptyset$. Then $B \cup \{e\}$ is extendible to a MST.

Correctness of MST algorithms

Theorem 2.5
For any connected graph G with arbitrary edge costs c, Prim's Algorithm finds a MST.

Theorem 2.6
For any connected graph G with arbitrary edge costs c, Kruskal's Algorithm finds a MST.

Efficiency of MST algorithms

How we store a connected graph $G = (V, E)$ in a computer:

- We keep a list of edges.
- An element of the list corresponding to edge $f = uv \in E$ is of the form (u, v, c_f) where c_f is the cost of f.

  Question: What is the length of this encoding?

Efficiency of MST algorithms

How we store a connected graph $G = (V, E)$ in a computer:

- We keep a list of edges.
- An element of the list corresponding to edge $f = uv \in E$ is of the form (u, v, c_f) where c_f is the cost of f.

  Question: What is the length of this encoding? $3m$

Efficiency of MST algorithms

How we store a connected graph $G = (V, E)$ in a computer:

- We keep a list of edges.
- An element of the list corresponding to edge $f = uv \in E$ is of the form (u, v, c_f) where c_f is the cost of f.

  Question: What is the length of this encoding? $3m$

Efficiency of Prim's algorithm

Prim's algorithm for MST

1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$ for some $r \in V$.
2. At each step add to T a least-cost edge $e \notin T$ such that H remains a tree.
3. Stop when H is spanning (tree).

Efficiency of Prim's algorithm

Prim's Algorithm can be restated using an observation from the proof of Theorem 2.5:

Prim's algorithm for MST

1. Initialize $H = (V(H), T)$ as $(\{r\}, \emptyset)$.
2. While H is not a (spanning) tree
   Add to T a minimum-cost edge from $\delta(V(H))$.

Efficiency of Prim's algorithm

Implementation of the "while loop":

- We keep $V(H)$ as a characteristic vector x
  (that is, $x_u = 1$ if $u \in V(H)$, and $x_u = 0$ if $u \notin V(H)$).

Efficiency of Prim's algorithm

Implementation of the "while loop":

- We keep $V(H)$ as a characteristic vector x (that is, $x_u = 1$ if $u \in V(H)$, and $x_u = 0$ if $u \notin V(H)$).
- At each step we run through E, checking for each $f = uv$ whether $f \in \delta(V(H))$ by checking whether $x_u \neq x_v$ and if so comparing c_f to the current minimum encountered.
  So e can be chosen in $O(m)$ time.

Efficiency of Prim's algorithm

Implementation of the "while loop":

- We keep $V(H)$ as a characteristic vector x (that is, $x_u = 1$ if $u \in V(H)$, and $x_u = 0$ if $u \notin V(H)$).
- At each step we run through E, checking for each $f = uv$ whether $f \in \delta(V(H))$ by checking whether $x_u \neq x_v$ and if so comparing c_f to the current minimum encountered. So e can be chosen in $O(m)$ time.
- Then x is updated by putting $x_v = 1$ where v is the end of e for which x_v was 0.

The number of steps performed in one "while loop" is $O(m)$.

Efficiency of Prim's algorithm

How many times is the "while" loop performed?

> Lemma 2.2
> A spanning connected subgraph of G is a spanning tree if and only if it has exactly $n-1$ edges.

Proof: Exercise

- Therefore the "while" loop is performed $n-1$ times.

So Prim's algorithm has a running time of $O(nm)$.

Note: Lemma 2.2 implies that every connected graph G with n nodes and m edges has $m \geq n-1$. Thus, in all complexity estimations we assume that $n = O(m)$ and that $m = O(n^2)$

Efficiency of Kruskal's algorithm

Kruskal's algorithm for MST

1. Initialize $H = (V, F)$ as $(V, \emptyset)$.
2. At each step add to F a least-cost edge $e \notin F$ such that H remains a (spanning) forest.
3. Stop when H is a (spanning) tree.

Efficiency of Kruskal's algorithm

- In Kruskal's algorithm, once an edge $e = vw$ becomes unavailable to add to F, that is, H contains a path from v to w, it remains so.

Kruskal's Algorithm can be restated:

Kruskal's algorithm for MST

1. Order E as $\{e_1, \ldots, e_m\}$, where $c_{e_1} \leq c_{e_2} \leq \cdots \leq c_{e_m}$.
2. Initialize $H = (V, F)$ as $(V, \emptyset)$.
3. For $i = 1, \ldots, m$
   If the ends of e_i are in different components of H
   Add e_i to F

Efficiency of Kruskal's algorithm

Kruskal's algorithm for MST

1. Order E as $\{e_1, \ldots, e_m\}$, where $c_{e_1} \leq c_{e_2} \leq \cdots \leq c_{e_m}$.
2. Initialize $H = (V, F)$ as $(V, \emptyset)$.
3. For $i = 1, \ldots, m$
   If the ends of e_i are in different components of H
   Add e_i to F

Phase 1: Sorting. Can be done in $O(m \log m)$.
Phase 2: We keep the partition of V into "blocks": the node-sets of components of H.

- $2m$ "finds": we find the block P containing a given v.
- $n - 1$ "merges": two blocks P, Q are replaced by $P \cup Q$, because edge uv with $u \in P$, $v \in Q$ has been added to F.

Efficiency of Kruskal's algorithm

How can we implement "find" and "merge"?

- We keep for each v the name block(v)
- each "find" can be done in 1 elementary step
- each "'merge" can be done by changing block(w) to block(v) for every w labelled block(u).
- Updating all the block(u) requires at most n elementary steps for a single merge.

The total work in the phase 2 of the algorithm is $O(m + n^2)$.

- We get a running time for Kruskal's Algorithm of $O(m \log m + m + n^2) = O(m \log m + n^2)$.